RELATÓRIO DE REFATORAÇÃO

Identificação e correção de Bad Smells

Disciplina:	Teste de Software
Trabalho:	Identificação e refatoração de Bad Smells
Aluno:	Guilherme Gomes de Brites
Matrícula:	808721

Projeto: bad-smells-js-refactoring

1. Análise de Bad Smells

Na análise manual do código original identifiquei três problemas principais no método generateReport().

1.1 Long Method (método longo)

O método reúne várias responsabilidades: gerar cabeçalho, processar itens, aplicar regras de acesso, calcular totais e montar o rodapé. Essa concentração dificulta a leitura, a localização de defeitos e os testes isolados de cada etapa.

1.2 Complex Conditional (excesso de condicionais)

As verificações de papel do usuário, formato do relatório e valor dos itens aparecem aninhadas. Isso aumenta a complexidade do fluxo e torna mais difícil testar combinações de cenários sem criar muitos casos de teste.

1.3 Duplicated Code (código duplicado)

A construção das linhas CSV e HTML é repetida em diferentes ramos para administradores e usuários comuns. A duplicação exige alterações em vários pontos e pode causar inconsistências entre os formatos ou perfis.

2. Relatório da ferramenta

Antes da refatoração, o comando npx eslint src/ apontou dois problemas:

Local	Regra / mensagem
11:3	sonarjs/cognitive-complexity — reduzir a complexidade cognitiva de 27 para o limite 15.
43:14	sonarjs/no-collapsible-if — combinar uma condição com a condicional interna.

O eslint-plugin-sonarjs complementou a revisão manual ao quantificar a complexidade cognitiva e apontar uma condicional simplificável. Esses alertas ajudam a identificar riscos de manutenção que podem não ser percebidos apenas pela leitura do código.

Evidência visual do ESLint



3. Processo de Refatoração

O smell escolhido como mais crítico foi Long Method, pois a concentração de responsabilidades também contribui para a complexidade condicional e para a duplicação.

Antes

```
generateReport(reportType, user, items) {
  let report = '';
  let total = 0;
  // Cabeçalho, regras de acesso, laço,
  // geração CSV/HTML, cálculo do total
  // e rodapé ficam no mesmo método.
}
```

Depois (estrutura refatorada)

```
  generateReport(reportType, user, items) {
    const visibleItems = this.filterItems(user, items);
    const total = visibleItems.reduce((sum, item) => sum + item.value, 0);

    return [
      this.generateHeader(reportType, user),
      this.generateBody(reportType, user, visibleItems),
      this.generateFooter(reportType, total),
    ].join('');
  }
```

Foi aplicada a técnica Extract Method: a geração do cabeçalho, corpo e rodapé foi separada em métodos específicos. Também foram separadas a filtragem dos itens e a geração de cada linha, reduzindo condicionais aninhadas e centralizando lógica repetida. A prioridade visual dos itens administrativos pode ser calculada na geração do HTML, evitando modificar diretamente os objetos de entrada.

4. Conclusão

Os testes automatizados funcionam como uma rede de segurança: verificam se o comportamento esperado continua válido após mudanças estruturais. A refatoração busca melhorar a organização interna sem alterar os resultados funcionais. Reduzir métodos longos, condicionais complexas e duplicação torna o software mais legível, testável e simples de manter.

Evidência após a refatoração

